

Mini-project in digital/computational pathology

Created by Pierre Cordier (DVM, PhD, Dipl.ECVP)
with the use of ChatGPT (GPT-5.6 Luna) - 01/10/2026

Mini-project: Build a WSI → patches → AI map pipeline

Beginner → early intermediate computational pathology

Environment: Google Colab | Main tools: Python, NumPy, pandas, matplotlib, OpenSlide, OpenCV/scikit-image, TIAToolbox, PyTorch

Learning objectives

- What a WSI pyramid is.
- How OpenSlide accesses a WSI.
- MPP, magnification and downsampling.
- Why WSIs are tiled.
- How to create a tissue mask.
- How to extract patches efficiently.
- How to preserve patch coordinates.
- How to visualize patches.
- What a pretrained pathology model actually receives.
- How patch-level predictions become a spatial WSI map.
- Why stain variation and domain shift matter.
- How to structure a small computational-pathology project in Python.

0. Project structure

```
project/
├── data/
│   ├── slides/
│   ├── results/
│   │   ├── thumbnails/
│   │   ├── masks/
│   │   ├── patches/
│   │   └── predictions/
│   ├── metadata/
│   │   └── patches.csv
│   └── notebooks/
│       └── computational_pathology_mini_project.ipynb
```

You don't have to physically reproduce this directory structure in Colab immediately. It's mainly teaching you how to organize a real project.

1. Know your computational environment

Exercise 1.1

```
import sys
import platform

print("Python:", sys.version)
print("Platform:", platform.platform())
```

Then

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("NumPy:", np.__version__)
print("pandas:", pd.__version__)
```

Questions

Q1. What Python version is actually running in Colab? Q2. Is it the same as Python 3.13.15 on your Mac?
Q3. Why is it useful to record package versions in a computational pathology project?

2. Install OpenSlide

Try

```
!pip install openslide-python
```

Then

```
import openslide

print(openslide.__version__)
```

If you get an error concerning the OpenSlide shared library

```
!apt-get update -qq
!apt-get install -y openslide-tools
```

Then

```
!pip install -q openslide-python

import openslide
print(openslide.__version__)
```

3. Get a small WSI

For the first run, use a public sample WSI, rather than your 500–1024 MB MRXS. TIAToolbox's documentation contains sample WSI material that is appropriate for experimentation. The important requirement is simply: obtain a small WSI format supported by OpenSlide, ideally .svs or .tif, and put it in Colab. You can upload it through Colab → Files → Upload or download a public test file. Don't worry about your .mrxs yet.

4. Your first WSI object

Suppose your file is sample.svs

```
import openslide

slide = openslide.OpenSlide("sample.svs")
print(slide.dimensions)
print(slide.level_count)
print(slide.level_dimensions)
print(slide.level_downsamples)
for key, value in slide.properties.items():
    print(key, ":", value)
```

Questions

Q4. What are the dimensions of level 0? How many pixels is that in total? Q5. How much larger is that than a normal 224×224 CNN input? Q6. Look at `slide.level_dimensions`. Why do the dimensions become progressively smaller? What do you think the pyramid is used for?

5. Understand the WSI pyramid

Level 0 100000×70000 Level 1 50000×35000 Level 2 25000×17500 Level 3 12500×8750 ...

Questions

Q7. Why would you use a low-resolution level to find tissue? Q8. Why wouldn't you normally use the lowest-resolution image to perform detailed nuclear classification? Q9. If level 1 has a downsample of 2 and level 3 has a downsample of 8, what does that mean?

6. Make your first thumbnail

```
thumbnail = slide.get_thumbnail((1500, 1500))
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))
plt.imshow(thumbnail)
plt.axis("off");
thumbnail.save("thumbnail.png")
```

Questions

Q10. Does the thumbnail preserve the full-resolution image? Q11. Why is it useful for tissue detection? Q12. What information is lost?

7. Understand `read_region()`

```
region = slide.read_region(
    location=(0, 0),
    level=0,
    size=(1000, 1000)
)

plt.figure(figsize=(8, 8))
plt.imshow(region)
plt.axis("off");

region2 = slide.read_region(
    location=(0, 0),
    level=2,
    size=(1000, 1000)
)

)
```

Questions

Q13. What is different between the two images? Q14. Does `size=(1000, 1000)` mean the same physical area at level 0 and level 2? Q15. Why is this distinction crucial when extracting pathology patches?

8. Understand MPP

Look through `slide.properties`. Search for properties related to MPP, MICRONS, OBJECTIVE, and MAGNIFICATION.

Exercise

```

mpp = 0.25
patch_size = 256

physical_size = mpp * patch_size

print(physical_size)

```

Questions

Q16. At $0.25\ \mu\text{m}/\text{pixel}$, what physical area does a 256×256 patch represent? Q17. At $0.5\ \mu\text{m}/\text{pixel}$? Q18. Why is saying merely "we used 256×256 patches" scientifically incomplete?

9. Build your own thumbnail-based tissue mask

```

img = np.array(thumbnail)

print(img.shape)
print(img.dtype)

```

Inspect RGB

```

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(img[:, :, 0], cmap="gray")
plt.title("Red")

plt.subplot(1, 3, 2)
plt.imshow(img[:, :, 1], cmap="gray")
plt.title("Green")

plt.subplot(1, 3, 3)
plt.imshow(img[:, :, 2], cmap="gray")
plt.title("Blue");

```

Questions

Q19. Why might the RGB channels behave differently for H&E? Q20. Which channel appears most useful for separating tissue from white background?

10. Simple thresholding

```

gray = img.mean(axis=2)

mask = gray < 220

plt.figure(figsize=(10, 10))
plt.imshow(mask, cmap="gray")
plt.axis("off");

```

Experiment with thresholds 180, 200, 220, and 240.

Questions

Q21. How does changing the threshold affect the mask? Q22. What happens if you choose a threshold that is too low? Q23. What happens if it is too high?

11. Improve the mask

```

!pip install -q opencv-python
import cv2

```

```
kernel = np.ones((5, 5), np.uint8)
```

```
mask_clean = cv2.morphologyEx(
    mask.astype(np.uint8),
    cv2.MORPH_CLOSE,
    kernel
)
```

Questions

Q24. What does morphological closing appear to do? Q25. Why might morphology be useful for tissue segmentation? Q26. What pathological artifacts could cause a simple thresholding approach to fail? Think about pale tissue, empty spaces, folds, pen marks, coverslip artifacts, necrosis, blood, and tissue fragments.

12. Connected components

```
num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
    mask_clean,
    connectivity=8
)
```

```
print("Number of components:", num_labels)
stats[:10]
```

Questions

Q27. Why might removing very small connected components improve your tissue mask? Q28. What assumption are you making when you remove small components? Q29. When could that assumption be wrong? Important pathology-AI lesson: Every preprocessing step contains assumptions about biology.

13. Compare your implementation with TIAToolbox

```
!pip install -q tiatoolbox
from tiatoolbox.wsicore.wsireader import WSIREader
wsi = WSIREader.open("sample.svs")
print(type(wsi))
```

Consult the current TIAToolbox documentation/examples for thumbnail generation, tissue masking, and patch extraction. Perform tissue masking with TIAToolbox and compare it with your implementation.

Questions

Q30. What did TIAToolbox automate that you previously implemented yourself? Q31. What advantages does a toolbox provide? Q32. What disadvantages might there be if you use a high-level function without understanding what it does?

14. Extract patches

Start conservatively: patch size 256×256 or 512×512 . Use a relatively low-resolution level initially so you aren't extracting enormous numbers of pixels. Your goal is ~100–500 patches, not hundreds of thousands.

First do it manually

```
x = 10000
y = 10000

patch = slide.read_region(
    (x, y),
    0,
    (512, 512))
```

```
)
```

```
patch.save("patch_001.png")
```

15. Build a patch extraction loop

Create a grid conceptually with `for y in range(...):` and `for x in range(...)`. Don't blindly extract everything. Use your tissue mask to determine whether the patch contains sufficient tissue.

A simple strategy is `tissue proportion = tissue pixels / patch pixels`. Keep the patch if tissue proportion

Important coordinate problem

Your mask might be 1500×1200 while your WSI is 100000×80000 . Therefore, mask coordinates and WSI coordinates aren't directly interchangeable. You need a scaling relationship.

Questions

Q33. Why can't you simply use `mask[y, x]` to decide whether WSI coordinate (x, y) contains tissue? Q34. How would you convert between thumbnail coordinates and level-0 coordinates? Q35. Why is it safer to use the actual WSI dimensions and thumbnail dimensions rather than assuming a fixed scale?

16. Build a patch metadata table

```
patches = []
patches.append({
    "patch_id": patch_id,
    "x": x,
    "y": y,
    "width": width,
    "height": height,
    "level": level,
    "tissue_fraction": tissue_fraction
})

df = pd.DataFrame(patches)

df.to_csv("patches.csv", index=False)
```

Questions

Q36. Why is the coordinate information just as important as the patch image? Q37. If you lose the coordinates, what becomes difficult or impossible later?

17. Make a patch gallery

Use matplotlib to create a grid of perhaps 25 patches and label each patch with its ID.

Pathologist's QC

For each gallery, identify tissue, background, artifact, potentially interesting regions, and poor-quality patches.

Questions

Q38. How many patches would you reject? Q39. Why? Q40. Could a machine-learning model learn something undesirable from the rejected patches if you included them?

18. Introduce stain normalization

Install/use TIAToolbox's stain normalization functionality. Apply at least one method to several patches. Compare Original → Normalized. Make a gallery. Then compare at least two methods if your environment allows it.

Questions

Q41. What visual changes do you observe? Q42. Why might stain normalization improve model robustness? Q43. Could normalization ever remove useful biological information? Q44. Why might a normalization method behave differently on veterinary tissue than on the human tissue it was developed/tested on?

19. Now introduce AI

Use a pretrained image encoder that can generate embeddings for your patches. Conceptually: patch → neural network → feature vector.

The vector might contain hundreds or thousands of numbers. This is a representation of the patch in a learned feature space.

20. Understand embeddings before classification

For example, a model might generate a 768-dimensional vector. For 500 patches the embedding matrix could have shape (500, 768).

Questions

Q45. What does one row represent? Q46. What does one column represent? Q47. Are the 768 features individually interpretable as "nucleus", "lymphocyte", "collagen", etc.? Be careful here.

21. Visualize the embeddings

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)

embedding_2d = pca.fit_transform(embeddings)

print(embedding_2d.shape)
plt.scatter(
    embedding_2d[:, 0],
    embedding_2d[:, 1]
)

plt.xlabel("PC1")
plt.ylabel("PC2");
```

Each dot is a pathology patch.

22. Color by your own annotation

Manually annotate perhaps 50–100 patches. Keep it simple: for example 0 = non-interesting/background and patch_id,label

001,0 002,1 003,0 ...

23. Train a very simple classifier

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
pred = model.predict(X_test)
from sklearn.metrics import classification_report

print(classification_report(y_test, pred))
```

24. Be very careful about train/test leakage

Suppose you have one WSI and randomly divide patches 80% train / 20% test. You may get an artificially impressive result because neighboring patches are extremely similar. Ideally, WSI 1 → training, WSI 2 → validation, WSI 3 → testing, or at least separate spatial regions.

Questions

Q48. Why is random patch-level splitting potentially problematic? Q49. Why is patient/animal-level splitting preferable? Q50. If you have only one WSI, can you legitimately claim that your model generalizes to other animals?

25. Create the AI map

You have patch_id, x, y, and prediction. Create a low-resolution canvas and place each patch's prediction at the corresponding location. Display it alongside the thumbnail.

26. Overlay predictions on the thumbnail

```
plt.imshow(thumbnail)
plt.imshow(prediction_map, alpha=0.5)
plt.axis("off");
```

The final figure should communicate: Where does the model think the interesting tissue is?

27. The most important experiment

Deliberately change patch size and MPP/resolution.

Experiment A: 256 × 256 at 0.25 $\mu\text{m}/\text{pixel}$. Experiment B: 512 × 512 at 0.5 $\mu\text{m}/\text{pixel}$. These may represent similar physical dimensions but very different pixel representations. Compare the resulting predictions.

Questions

Q51. Does the model's behavior change with scale? Q52. Why might cellular morphology require a different resolution than tissue architecture? Q53. What biological structures become visible/disappear when changing resolution?

28. Domain shift: human → veterinary

Suppose your pretrained model was trained on human H&E and you apply it to canine, feline, or equine H&E.

Questions

Q54. Why might the model still work? Q55. Why might it fail? Think about staining, scanner, tissue preparation, species, anatomy, disease morphology, and prevalence.

29. MRXS: move to your own data

Take one anonymized MRXS slide. Don't start with 100 slides.

```
slide = openslide.OpenSlide("your_slide.mrxs")

print(slide.dimensions)
print(slide.level_count)

for key, value in slide.properties.items():
    print(key, value)

thumbnail = slide.get_thumbnail((1500, 1500))
plt.imshow(thumbnail)
plt.axis("off");
```

30. Compare your own MRXS to the sample WSI

Create a little table with: Property, Public WSI, Your MRXS; then fill in Vendor, Format, Dimensions, Pyramid levels, MPP, Objective, File size, and H&E?.

31. Investigate your MRXS metadata

```
slide.properties
slide.associated_images
```

Questions

Q56. What metadata does your scanner provide? Q57. Does the WSI contain information about MPP? Q58. Is the MPP the same in X and Y? Q59. What scanner produced the slide? Q60. Are there associated images such as macro/label images?

32. A useful engineering exercise

```
def inspect_wsi(path):
    ...
```

It should print File, Format, Dimensions, Levels, MPP-X, MPP-Y, and Objective. Then call `inspect_wsi("slide.mrxs")`. This is your first reusable pathology utility.

33. Make a reusable tissue-mask function

```
def make_tissue_mask(slide, ...):
    ...
    return mask

mask = make_tissue_mask(slide)
```

34. Make a reusable patch extractor

```
patches = extract_patches(
    slide,
    tissue_mask,
    patch_size=512,
    level=0
)
```

with output such as images/ and metadata/. This is the point at which you're beginning to write a small computational pathology library rather than a one-off notebook.

35. Your final deliverable

Figure 1 — WSI thumbnail: whole slide

Figure 2 — Tissue mask: tissue vs background

Figure 3 — Patch grid: 100–500 extracted patches

Figure 4 — Patch embedding PCA: 2D representation of patch embeddings

Figure 5 — AI predictions: patch-level predictions

Figure 6 — WSI overlay: thumbnail + AI prediction map

One table — WSI metadata

Final questions — see whether you've really understood the pipeline

- Q61. Why can't we simply load a 1 GB WSI using PIL and process it as a normal image?
- Q62. What is a WSI pyramid?
- Q63. What is the difference between pixel resolution and physical resolution?
- Q64. What is MPP?
- Q65. Why is a 256×256 patch not enough information to describe a pathology patch?
- Q66. What is a tissue mask?
- Q67. Why do we create it?
- Q68. Why should patch coordinates be stored?
- Q69. What is the difference between a patch and an embedding?
- Q70. What does a pretrained model give us?
- Q71. Why might using a pretrained human pathology model on veterinary tissue be problematic?
- Q72. What is domain shift?
- Q73. Why is patient/animal-level splitting preferable to random patch-level splitting?
- Q74. Why might your model have excellent accuracy while being scientifically useless?
- Q75. Why might a pathologist prefer a heatmap over a list of predictions?
- Q76. Why is spatial information important?
- Q77. What happens if you increase patch size?
- Q78. What happens if you increase MPP?
- Q79. What information is lost when going from a WSI to a thumbnail?
- Q80. What information is lost when going from a patch to a single classification label?

Optional challenge: turn it into a real mini-study

Once the basic pipeline works, take 3–5 veterinary WSIs rather than one. For each: WSI → tissue segmentation → patch extraction → embeddings → PCA/UMAP → clustering. Then ask: Do patches

naturally cluster according to morphological tissue compartments?

You could manually inspect clusters and discover things like: Cluster A → adipose; Cluster B → collagen/stroma; Cluster C → cellular tissue; Cluster D → blood; Cluster E → artifact. This would introduce you to unsupervised representation learning, an excellent next step for someone with your pathology background.

Suggested progression

Milestone	What you learn
1. WSI inspection	OpenSlide, pyramids, metadata
2. Thumbnail	resolution and memory
3. Tissue mask	NumPy + image processing
4. Patch extraction	coordinates + tiling
5. Patch QC	pathology + computational QC
6. TIAToolbox	high-level WSI APIs
7. Embeddings	deep-learning representations
8. PCA/UMAP	dimensionality reduction
9. Simple classifier	supervised ML
10. Spatial map	WSI-level inference
11. MRXS	your actual scanner/data
12. Veterinary domain shift	research-level thinking

What I would not learn yet

- writing your own CNN
- segmentation networks
- MIL
- transformers from scratch
- distributed training
- Docker
- Kubernetes
- massive datasets
- complex annotation formats
- whole-slide model training
- foundation-model fine-tuning

Those are all useful eventually, but they'll make the learning curve unnecessarily steep.

Final recommendation

Because you are a veterinary pathologist, keep a pathology notebook alongside the coding notebook. For every experiment, record not only patch size, MPP, and model, but also: What morphological scale am I asking the model to see? What tissue compartments are represented? What artifacts are present? What could constitute leakage? What biological assumptions am I making?

The goal is to become comfortable with this sentence: “I have a WSI; I know how its pyramid is organized; I can access arbitrary regions at a defined physical resolution; I can identify tissue; I can extract spatially registered patches; I can turn patches into representations; and I can put model predictions back into their spatial context.”

Once that sentence is genuinely intuitive, TIAToolbox, OpenSlide, PyTorch, pathology foundation models, MIL, segmentation, and WSI classification all become much easier to learn.